Tutorial V - Production Planning in Breweries

Applied Optimization with Julia



After this tutorial, you will be able to implement the Capacitated Lot-Sizing Problem (CLSP) from the lecture in JuMP, work with periods that are stored as strings, and diagnose a model that looks fine at first glance but ignores part of the demand.

> **Optional: A Zed Theme for the Tutorials**
> In these tutorial files, the comments are the instructions, yet most Zed themes grey them out. The theme Paper & Ink does the opposite: it gives the comments the strongest contrast and the code calmer colors. Run the next cell once to install it. Then quit Zed completely (macOS: `Cmd+Q`), start it again, open the command palette (`Ctrl+Shift+P`, macOS `Cmd+Shift+P`), run **theme selector: toggle** and choose **Paper & Ink Light** or **Paper & Ink Dark**.

```
using Downloads
themes = Sys.iswindows() ? joinpath(ENV["APPDATA"], "Zed", "themes") :
expanduser("~/.config/zed/themes")
mkpath(themes)
Downloads.download("https://raw.githubusercontent.com/beyondsimulations/paper-and-ink-theme/main/themes/paper-and-ink.json", joinpath(themes, "paper-and-ink.json"))
```

1. Modelling the CLSP

Load the necessary packages and data

Implement the CLSP from the lecture in Julia. Before we start, let's load the necessary packages and data.

```
using JuMP, HiGHS
using CSV
using DataFrames
using Plots
using StatsPlots
plotly() # This will create interactive plots later on
```

> **Tip**
> If you haven't installed the packages yet, you can do so by running `using Pkg` first and then `Pkg.add(["JuMP", "HiGHS", "CSV", "DataFrames", "Plots", "StatsPlots"])`.

> **Note**
> Zed cannot show interactive plots yet. If you work in Zed, remove the `plotly()` line: your plots then appear as images.

Load the data

Now, let's load the data. The weekly demand in bottles $d_{i,t}$, the available time at the bottling plant in hours a_t, the time required to bottle each beer in hours b_i, and the setup time in hours g_i are provided as CSV files.

```
# Get the directory of the current file
file_directory = "$(@__DIR__)/data"

# Load the data about the available time at the bottling plant
availableTime = CSV.read("$file_directory/availabletime.csv", DataFrame)
println("Number of periods: $(nrow(availableTime))")
println("First 5 rows of available time per period:")
println(availableTime[1:5, :])
```

```
Number of periods: 27
First 5 rows of available time per period:
5×2 DataFrame
 Row │ period     available_capacity
     │ DataStri…  Int64
─────┼──────────────────────────────
   1 │ week_01                  168
   2 │ week_02                  168
   3 │ week_03                  168
   4 │ week_04                  168
   5 │ week_05                   48
```

> **Note**
> Take a closer look at `availabletime.csv`: in `week_05`, the plant only has 48 hours available instead of the usual 168. Bottling the demand of that week alone would take roughly 102 hours of pure bottling time. What does that mean for the weeks before `week_05`, and what do you expect to see in the warehouse plot later on?

```
# Load the data about the bottling time for each beer
bottlingTime = CSV.read("$file_directory/bottlingtime.csv", DataFrame)
println("Number of beers: $(nrow(bottlingTime))")
println("Bottling time per beer:")
println(bottlingTime)
```

```
Number of beers: 6
Bottling time per beer:
```

```
6×2 DataFrame
 Row │ beer_type   bottling_time
     │ DataStri…   Float64
─────┼──────────────────────────
   1 │ Pilsener          0.00222
   2 │ Blonde_Ale        0.00111
   3 │ Amber_Ale         0.00139
   4 │ Brown_Ale         0.00222
   5 │ Porter            0.00167
   6 │ Stout             0.00111
```

```
# Load the data about the setup time for each beer
setupTime = CSV.read("$file_directory/setuptime.csv", DataFrame)
println("Setup time per beer:")
println(setupTime)
```

```
Setup time per beer:
6×2 DataFrame
 Row │ beer_type   setup_time
     │ DataStri…   Int64
─────┼────────────────────────
   1 │ Pilsener            10
   2 │ Blonde_Ale          11
   3 │ Amber_Ale            8
   4 │ Brown_Ale            8
   5 │ Porter              11
   6 │ Stout                9
```

```
# Load the data about the weekly demand for each beer
demandCustomers = CSV.read("$file_directory/demand.csv", DataFrame)
println("First 5 rows of demand per beer:")
println(demandCustomers[1:5, :])
```

```
First 5 rows of demand per beer:
5×3 DataFrame
 Row │ beer_type   period     demand
     │ DataStri…   DataStri…  Int64
─────┼──────────────────────────────
   1 │ Pilsener    week_01      3853
   2 │ Blonde_Ale  week_01      8372
   3 │ Amber_Ale   week_01     16822
   4 │ Brown_Ale   week_01     13880
   5 │ Porter      week_01     10642
```

Define the parameters

Consider in your implementation, that **each hour of setup** is associated with a cost of 1000 Euros, and the inventory holding cost for unsold bottles at the end of each period is 0.1 Euro per bottle. Implement **both parameters** for the cost of setup and the inventory holding cost in the model. Call them `setupHourCosts` and `warehouseCosts`.

```
# YOUR CODE BELOW


# Validate your solution
@assert setupHourCosts > 0 "Setup hour costs must be positive"
@assert warehouseCosts > 0 "Warehouse costs must be positive"
@assert setupHourCosts == 1000 "Setup hour costs must be 1000 euros"
@assert warehouseCosts == 0.1 "Warehouse costs must be 0.1 euros per bottle"
```

Next, you need to prepare the given data for the model. Create a dictionary for the available time, bottling time, and setup time. Call them `dictAvailableTime`, `dictBottlingTime`, and `dictSetupTime`.

> **Note**
> Let's understand what's happening with `dictDemand` in the code. It creates a dictionary where:
>
> - **Keys** are tuples `(beer_type, period)`, e.g., `("Pilsener", "week_01")`
> - **Values** are the demand numbers for that beer in that period
>
> You can use this pattern to create dictionaries for:
>
> - `dictAvailableTime`: Maps each **period** (single key) → available time
> - `dictBottlingTime`: Maps each **beer type** (single key) → bottling time per bottle
> - `dictSetupTime`: Maps each **beer type** (single key) → setup time
>
> Notice that these dictionaries have **single keys** (just the period or just the beer type), not tuple keys like `dictDemand`.

```
# Prepare the data for the model
dictDemand = Dict((row.beer_type,row.period) => row.demand for row in
eachrow(demandCustomers))

# YOUR CODE BELOW


# Validate your solution
@assert length(dictAvailableTime) == nrow(availableTime) "Available time
dictionary should have same length as input data"
@assert length(dictBottlingTime) == nrow(bottlingTime) "Bottling time
dictionary should have same length as input data"
@assert length(dictSetupTime) == nrow(setupTime) "Setup time dictionary should
have same length as input data"
```

```
# Check that all values are positive
@assert all(v -> v > 0, values(dictAvailableTime)) "All available time values must be positive"
@assert all(v -> v > 0, values(dictBottlingTime)) "All bottling time values must be positive"
@assert all(v -> v > 0, values(dictSetupTime)) "All setup time values must be positive"

# Check that dictionaries contain all expected keys
@assert all(p -> haskey(dictAvailableTime, p), availableTime.period) "Missing periods in available time dictionary"
@assert all(b -> haskey(dictBottlingTime, b), bottlingTime.beer_type) "Missing beer types in bottling time dictionary"
@assert all(b -> haskey(dictSetupTime, b), setupTime.beer_type) "Missing beer types in setup time dictionary"
```

Define the model instance

Next, we define the model instance for the CLSP.

```
# Prepare the model instance
lotsizeModel = Model(HiGHS.Optimizer)
set_attribute(lotsizeModel, "presolve", "on")
set_time_limit_sec(lotsizeModel, 60.0)
```

Define the variables

Now, create your variables. Please name them `productBottled` for the binary setup variable (it is 1 if a beer type is bottled in a period, and 0 otherwise), `productQuantity` for the production quantity and `WarehouseStockPeriodEnd` for the warehouse stock at the end of each period. We will use these names later in the code to plot the results.

Note that we model the production quantity and the warehouse stock as continuous variables, although you cannot bottle half a bottle. With quantities this large, the error from allowing fractional bottles is negligible and the model is much easier to solve.

> **Tip**
> When creating your variables, you'll have to create one variable for **each combination** of beer type and period. You can use the dictionary keys directly in the variable definition:
>
> ```
> @variable(model, x[keys(dictBottlingTime), keys(dictAvailableTime)] >= 0)
> ```
>
> This creates variables indexed by:
>
> - First index: beer types (from `dictBottlingTime`)
> - Second index: periods (from `dictAvailableTime`)
>
> So you'll have variables like `x["Pilsener", "week_01"]`, `x["Pilsener", "week_02"]`, etc.
>
> Don't forget the variable domains: `productBottled` needs `Bin`, while `productQuantity` and `WarehouseStockPeriodEnd` need the lower bound `>= 0`. Without the lower bounds, the solver could store a *negative* number of bottles, which makes the model unbounded.

```
# YOUR CODE BELOW
```

```
# Validate your solution
# Check if variables exist in the model
@assert haskey(lotsizeModel.obj_dict, :productBottled) "productBottled variable
not found in model"
@assert haskey(lotsizeModel.obj_dict, :productQuantity) "productQuantity
variable not found in model"
@assert haskey(lotsizeModel.obj_dict, :WarehouseStockPeriodEnd)
"WarehouseStockPeriodEnd variable not found in model"

# Check variable dimensions
@assert length(productBottled) == length(dictBottlingTime) *
length(dictAvailableTime) "Incorrect dimensions for productBottled"
@assert length(productQuantity) == length(dictBottlingTime) *
length(dictAvailableTime) "Incorrect dimensions for productQuantity"
@assert length(WarehouseStockPeriodEnd) == length(dictBottlingTime) *
length(dictAvailableTime) "Incorrect dimensions for WarehouseStockPeriodEnd"

# Check variable types and domains
@assert all(is_binary, productBottled) "productBottled must be binary
variables"
@assert !any(is_integer, productQuantity) "productQuantity must be continuous
variables"
@assert !any(is_integer, WarehouseStockPeriodEnd) "WarehouseStockPeriodEnd must
be continuous variables"
@assert all(v -> has_lower_bound(v) && lower_bound(v) == 0, productQuantity)
"productQuantity needs the lower bound >= 0"
```

```
@assert all(v -> has_lower_bound(v) && lower_bound(v) == 0,
WarehouseStockPeriodEnd) "WarehouseStockPeriodEnd needs the lower bound >= 0"
```

Define the objective function

Next, define the objective function.

> **Tip**
> The objective function needs to sum costs across **all beer types** and **all periods**. With dictionaries, you reference the data like this:
>
> ```
> @objective(model, Min,
>     sum(... for i in keys(dictBottlingTime), t in keys(dictAvailableTime))
> )
> ```
>
> Inside the sum, you'll need:
>
> 1. **Setup costs**: `setupHourCosts * dictSetupTime[i] * productBottled[i,t]`
>   - This uses `dictSetupTime[i]` to get the setup time for beer type `i`
> 2. **Inventory holding costs**: `warehouseCosts * WarehouseStockPeriodEnd[i,t]`
>
> Notice how we access dictionary values using `dict[key]` notation!

```
# YOUR CODE BELOW


# Validate your solution
# Check if the model has an objective
@assert objective_function(lotsizeModel) !== nothing "Model must have an
objective function"

# Check if it's a minimization problem
@assert objective_sense(lotsizeModel) == MOI.MIN_SENSE "Objective should be
minimization"

# Check if the objective function contains both cost components
obj_expr = objective_function(lotsizeModel)
@assert contains(string(obj_expr), "productBottled") "Objective must include
setup costs (productBottled)"
@assert contains(string(obj_expr), "WarehouseStockPeriodEnd") "Objective must
include warehouse costs (WarehouseStockPeriodEnd)"
```

Define the constraints

Now, we need to define all necessary constraints for the model. Start with the demand/inventory balance constraint.

> **Tip**
> The first period is special, as it does not have a previous period. Furthermore, we are working with strings as variable references, thus we cannot use `t-1` directly as in the lecture. To address this, we could collect and sort all keys and then use their indices to address the previous period. For example, `all_periods[t-1]` would then be the previous period, if we index t just as a range from `2:length(all_periods)`.
>
> Note that sorting the period strings only works here because the period names are zero-padded (`week_01`, …, `week_27`). Without the padding, `week_10` would be sorted right after `week_1`!

```
# Get the first period and all periods
first_period = first(sort(collect(keys(dictAvailableTime))))
all_periods = sort(collect(keys(dictAvailableTime)))
```

With these, we can now define the demand/inventory balance constraint. As this is the first constraint and might be a bit tricky, the solution is already given below.

```
# Inventory balance constraints for periods after first period
@constraint(lotsizeModel,
    demandBalance[i=keys(dictBottlingTime), t=2:length(all_periods)],
    WarehouseStockPeriodEnd[i,all_periods[t-1]] +
productQuantity[i,all_periods[t]] - WarehouseStockPeriodEnd[i,all_periods[t]]
== dictDemand[i,all_periods[t]]
    )
```

Next, we need to ensure that a beer type can only be bottled in a period if the plant is set up for it in that period. This is the “Big-M” constraint from the lecture:

$$X_{i,t} \leq Y_{i,t} \times \sum_{\tau \in \mathcal{T}} d_{i,\tau} \quad \forall i \in \mathcal{I}, t \in \mathcal{T}$$

```
# YOUR CODE BELOW
```

```
# Validate your solution
@assert num_constraints(lotsizeModel, AffExpr, MOI.LessThan{Float64}) ==
length(dictBottlingTime) * length(dictAvailableTime) "You need one setup
constraint for each combination of beer type and period"
```

Last, we need to limit the time used at the bottling plant. In each period, the total bottling time **plus the setup time** of all bottled beer types must not exceed the available time:

$$\sum_{i \in \mathcal{I}} (b_i \times X_{i,t} + g_i \times Y_{i,t}) \leq a_t \quad \forall t \in \mathcal{T}$$

```
# YOUR CODE BELOW
```

```
# Validate your solution
@assert num_constraints(lotsizeModel, AffExpr, MOI.LessThan{Float64}) ==
length(dictBottlingTime) * length(dictAvailableTime) +
length(dictAvailableTime) "You need one capacity constraint for each period (in
addition to the setup constraints)"

# Each capacity constraint has to contain the quantity and the setup of every
beer type
all_lt_constraints = all_constraints(lotsizeModel, AffExpr,
MOI.LessThan{Float64})
number_capacity_constraints = count(c ->
length(constraint_object(c).func.terms) == 2 * length(dictBottlingTime),
all_lt_constraints)
@assert number_capacity_constraints == length(dictAvailableTime) "Each capacity
constraint must include the bottling time AND the setup time of every beer
type"
```

Solve the model

Finally, implement the solve statement for your model instance.

```
# YOUR CODE BELOW


# Validate your solution
@assert primal_status(lotsizeModel) == MOI.FEASIBLE_POINT "The solver found no
feasible solution within the time limit - check your model and re-run the cell"
@assert 360000 <= objective_value(lotsizeModel) <= 900000 "Objective value
should be between 360,000 and 900,000"
```

Now, unfortunately we cannot assert the value of the objective function perfectly here, as the model is hard to solve and we abort the computation at the time limit of 60 seconds. The solver then returns the best solution found so far, so everybody is likely getting slightly different results. The solution for the first task will likely be in the **range of 550,000 to 700,000**, and the validation above is deliberately generous to allow for this variation. Don't worry if the solver reports a large remaining gap — the best solution found is good enough for our purposes.

Create the plots

The following code creates production and warehouse plots for you. Use it to verify and visualize your solution in the following tasks.

> **Note**
> The creation of the dataframes and the plots is implemented inside a function, as we will need to use it multiple times in the following tasks.

```
# Create the production results
function create_production_results()
    # Create a DataFrame to store the results
    productionResults = DataFrame(
        period = String[],
        product = String[],
        productBottled = Bool[],
        productQuantity=Int[],
        WarehouseStockPeriodEnd=Int[]
    )

    # Populate the DataFrame with the results
    for i in keys(dictSetupTime)
        for t in keys(dictAvailableTime)
            push!(
                productionResults,(
                period = t,
                product = i,
                productBottled = value(productBottled[i,t]) > 0.5,
                productQuantity = round(Int,value(productQuantity[i,t])),
                WarehouseStockPeriodEnd =
round(Int,value(WarehouseStockPeriodEnd[i,t])),
                )
            )
        end
    end

    sort!(productionResults,[:period, :product])
    return productionResults
end

# Create the production plot
function create_production_plot(productionResults)
    p = groupedbar(
        productionResults.period,
        productionResults.productQuantity,
        group=productionResults.product,
        ylabel="Production Quantity (Bottles)",
        xlabel="Period",
        title="Production Schedule by Beer Type",
        size=(1200,600),
        palette = :Set3,
        legend=:outertopright,
        xrotation = 45,
        legendtitle="Beer Type",
        bar_width=0.7,
        grid=false,
        dpi=300
```

```
    )
    return p
end

# Create the warehouse stock plot
function create_warehouse_plot(productionResults)
    p = groupedbar(
        productionResults.period,
        productionResults.WarehouseStockPeriodEnd,
        group=productionResults.product,
        ylabel="Warehouse Stock",
        xlabel="Period",
        title="Warehouse Stock",
        size=(1200,600),
        palette = :Set3,
        legend=:outertopright,
        xrotation = 45,
        legendtitle="Beer Type",
        bar_width=0.7,
        grid=false,
        dpi=300
    )
    return p
end
```

The following code creates the production plot.

```
productionResults = create_production_results()
p = create_production_plot(productionResults)
```

The following code creates the warehouse stock plot. Note that we can reuse the `productionResults` DataFrame from the previous cell.

```
p = create_warehouse_plot(productionResults)
```

Calculate the setup and inventory costs

Next, we calculate the setup and inventory costs for each period and store them in a DataFrame. This should also work for you, if you followed the previous name instructions.

```
# Calculate costs per period
function create_cost_results()
    costResults = DataFrame(
        period = String[],
        setup_costs = Float64[],
        inventory_costs = Float64[]
```

```
    )

    for t in sort(collect(keys(dictAvailableTime)))
        # Calculate setup costs for this period
        # We round the binary values, as the solver only guarantees them up to
a small tolerance
        period_setup_costs = sum(
            setupHourCosts * dictSetupTime[i] *
round(value(productBottled[i,t]))
            for i in keys(dictBottlingTime)
        )

        # Calculate inventory costs for this period
        period_inventory_costs = sum(
            warehouseCosts * value(WarehouseStockPeriodEnd[i,t])
            for i in keys(dictBottlingTime)
        )

        push!(costResults, (
            period = t,
            setup_costs = period_setup_costs,
            inventory_costs = period_inventory_costs
        ))
    end

    # Stack the cost columns
    stacked_costs = stack(costResults, [:setup_costs, :inventory_costs],
                          variable_name="Cost_Type", value_name="Cost")
    return stacked_costs
end

# Create the cost plot
function create_cost_plot(stacked_costs)
    p = groupedbar(
        stacked_costs.period,
        stacked_costs.Cost,
        group=stacked_costs.Cost_Type,
        ylabel="Costs (€)",
        xlabel="Period",
        title="Setup and Inventory Costs per Period",
        size=(1200,600),
        palette=:Set2,
        legend=:outertopright,
        xrotation=45,
        legendtitle="Cost Type",
        bar_width=0.7,
        grid=false,
        dpi=300
```

```
    )
    return p
end
```

The following code calls the setup and inventory costs plot.

```
stacked_costs = create_cost_results()
p = create_cost_plot(stacked_costs)
```

---

2. Initial Warehouse Stock

Take a closer look at your production plot: most likely, nothing is bottled in `week_01` at all! Our model so far has no inventory balance constraint for the first period. The solver can therefore "invent" free starting stock `WarehouseStockPeriodEnd[i, "week_01"]` out of nothing, and the demand of the first week is never enforced.

Fix this by adding the **inventory balance for the first period**, assuming that the warehouse starts empty. As in the lecture, we set the initial stock $W_{i,0} = 0$, so the balance for the first period becomes:

$$X_{i,1} - W_{i,1} = d_{i,1} \quad \forall i \in \mathcal{I}$$

To solve this task, you can simply extend the previous model by these additional constraints in the cell below. The variable `first_period`, which we defined earlier, is exactly what you need here. Afterwards, re-run the optimization.

```
# YOUR CODE BELOW


# Validate your solution
# A balance constraint links at least two variables - fixing a single variable
to zero is not enough
balance_constraints = all_constraints(lotsizeModel, AffExpr,
MOI.EqualTo{Float64})
number_balance_constraints = count(c -> length(constraint_object(c).func.terms)
>= 2, balance_constraints)
@assert number_balance_constraints == length(dictBottlingTime) *
length(dictAvailableTime) "You need one inventory balance constraint for each
combination of beer type and period - did you add the balance for the first
period?"
@assert primal_status(lotsizeModel) == MOI.FEASIBLE_POINT "The solver found no
feasible solution within the time limit - check your model and re-run the cell"
@assert 500000 <= objective_value(lotsizeModel) <= 950000 "Objective value
should be between 500,000 and 950,000"
```

The objective value should now be higher, likely in the **range of 700,000 to 800,000**. The solution space is smaller than before: the model can no longer invent starting stock,

but has to bottle the demand of the first week itself. You can check the plots for the production and warehouse stock to verify this.

```
productionResults = create_production_results()
p = create_production_plot(productionResults)
```

```
p = create_warehouse_plot(productionResults)
```

```
stacked_costs = create_cost_results()
p = create_cost_plot(stacked_costs)
```

---

3. Scheduled Repair

Unfortunately, the bottling plant has to undergo maintenance in periods `"week_10"` and `"week_11"`. **Extend your model** to prevent any production in those two periods.

Again, to solve this task, you can simply extend the previous model by these additional constraints in the cell below. Afterwards, you can re-run the optimization.

```
# YOUR CODE BELOW
```

```
# Validate your solution
@assert primal_status(lotsizeModel) == MOI.FEASIBLE_POINT "The solver found no feasible solution within the time limit - check your model and re-run the cell"
@assert 520000 <= objective_value(lotsizeModel) <= 950000 "Objective value should be between 520,000 and 950,000"
@assert all(value(productQuantity[i,t]) <= 1e-6 for i in keys(dictBottlingTime), t in ["week_10", "week_11"]) "No production is allowed in week_10 and week_11"
```

Again, the solution space is smaller, so the true optimal objective value can only increase. Because we stop the solver at the time limit, the value you see can still be a bit lower than before — in that case, the solver simply found a better solution for the restricted model than it previously did for the unrestricted one. You can check the plots for the production and warehouse stock to verify whether the production is zero in the maintenance periods.

```
productionResults = create_production_results()
p = create_production_plot(productionResults)
```

```
p = create_warehouse_plot(productionResults)
```

```
stacked_costs = create_cost_results()
p = create_cost_plot(stacked_costs)
```

---

4. Production Schedule Analysis

Analyze the production schedule of your current model, which includes the constraints from sections 2 and 3. Is the workload **distributed evenly** across all time periods? Provide a rationale for your assessment.

Please answer in the following cell. Note that `#=` and `=#` are comment delimiters for multiline comments. You can write whatever you want between them and the code will not be executed.

```
# YOUR REASONING BELOW
#=



=#
```

Based on the production data from the final period, **calculate the ending inventory levels for each type of beer**. Discuss any significant findings. Compute the ending inventory levels for each type of beer in the following cell. You can name the DataFrame however you want.

```
# YOUR CODE BELOW
```

---

5. Semiannual Bottling Strategy

Reflecting on a scenario where the company schedules its bottling operations **semiannually (every six months)** using the current method: identify and discuss potential pitfalls of this strategy.

Offer at least one actionable suggestion for enhancing the efficiency or effectiveness of the production planning process.

```
# YOUR ANSWER BELOW
#=



=#
```

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography